

A C++ Implementation of the GAFF2 Force Field for Evaluation of Small Organic Molecules

Abstract

Classical force fields evaluate molecular potential energies at orders of magnitude lower runtimes than *ab initio* methods. Here, we implement four out of five energy terms for version 2 of the General AMBER Force Field (GAFF2), describing bond stretching, angle bending, dihedral torsion, and van der Waals interactions. Our code takes atomic coordinates as its sole input, infers covalent connectivity from interatomic distances, and assigns AMBER atom types across 17 GAFF2 types. We benchmark calculated energies against AmberTools 25 for a variety of molecules, including hydrocarbons, alcohols, carboxylic acids, amines, haloalkanes, and simple amino acids. Strong agreement across the different classes of molecules suggests a proper implementation of the force field.

Background

Classical force fields describe interatomic interactions through a parametrized potential energy equation. Modern implementations of force fields exhibit a typical runtime scaling of $O(N)$ to $O(N \log \log N)$, where N is the number of atoms in the molecule or material being modeled.¹ In contrast, Kohn-Sham density functional theory (DFT) scales as $O(N^3)$, as we learned in lecture. Thus, while force fields may not offer the same accuracy as *ab initio* methods, they offer advantageous runtimes for large systems such as biological molecules or proteins. In addition, recent machine-learned force fields aim to close the gap in accuracy by using neural-network parametrizations of the potential surface to approach DFT-level accuracy at runtime cost approaching that of classical force fields.²

Classical force fields fall broadly into three classes of complexity, distinguished by the physics described in their functional form. Class 1 force field (e.g., AMBER) describe bonded interactions through simple harmonic stretching and bending terms, a periodic torsion potential, and pair-additive van der Waals interactions.³ Class 2 force fields (e.g., MMFF94) augment Class 1 models with anharmonic stretching corrections and explicit cross-coupling terms between adjacent bonds, angles, and torsions.⁴ Lastly, Class 3 force fields (e.g., AMOEBA) introduce specific chemical terms to often describe phenomena in organic chemistry, such as inductive effects.⁵ In this project, we implement four out of five energy terms in GAFF2, a Class 1 force field.

The AMBER force field describes the molecular potential energy as a sum of bonded and nonbonded contributions. Each energy term in the force field is as follows:

- Harmonic bond stretching
- Harmonic angle bending
- Torsion potential
- Lennard-Jones 12-6 dispersion-repulsion term
- Electrostatic potential

The electrostatic term requires per-atom partial charges that are not bundled with the GAFF2 parameter distribution, as they are typically derived separately via AM1-BCC or RESP fitting against quantum-mechanical reference data. Thus, we omit the Coulomb term in our implementation and report only the four bonded and dispersive contributions.

Our implementation takes a molecular geometry in XYZ format as the only input needed from the user. Covalent bonds are inferred from pairwise interatomic distances against a tabulated reference, and AMBER atom types are assigned through a four-stage algorithm based on element identity and bonding environment. The 17 supported GAFF2 atom types cover alkanes, alkenes, alkynes, alcohols, primary amines, carboxylic acids, haloalkanes (Cl, Br, F substituents), and simple amino acids.

Objectives

1. Implement the four AMBER energy terms (bond stretching, angle bending, dihedral torsion, and van der Waals) in C++.
2. Build a parameter-lookup pipeline based on AMBER atom type.

3. Represent molecules as graphs to enable enumeration of pairs, triplets, and quadruplets required for parameter assignment.
4. Develop an atom typing pipeline covering 17 GAFF2 types relevant to small organic molecules.
5. Benchmark energy calculations against AmberTools reference calculations.

Methodology

Functional form. The total energy is described as:

$$E_{total} = E_{bond} + E_{angle} + E_{dihedral} + E_{vdW} \quad (1)$$

with the four contributions defined in equations 2-5 below. Bond stretching is modeled harmonically about an equilibrium bond length r_{eq} ,

$$E_{bond} = \sum_{bonds} K_r (r - r_{eq})^2 \quad (2)$$

Where K_r is the spring constant, r_{eq} is the equilibrium, length, and r is the distance between the pair of atoms. The specific values for the parameters K_r and r_{eq} are based on the atom types of the bonded atoms.

Angle bending follows the same harmonic approximation about an equilibrium angle θ_{eq} ,

$$E_{angle} = \sum_{angles} K_{\theta} (\theta - \theta_{eq})^2 \quad (3)$$

Where the specific values for the parameters K_{θ} and θ_{eq} are based on the atom types of the triplet of atoms making up the angle. Dihedral torsion is represented as a truncated Fourier series,

$$E_{dihedral} = \sum_{dihedrals} \sum_n \frac{V_n}{d} [1 + \cos(n\phi - \gamma)], \quad (4)$$

where V_n is the barrier height for the n -fold component, d is an integer divider, γ is the phase offset, and n is the periodicity. Dihedral parameters are based on the atom types of the quadruplet defining the rotation. In particular, the wildcard token “X” may be used at the terminal positions to enable lookups against torsions whose central bond is the only well-defined feature. Lastly, pairwise dispersion and repulsion follow the Lennard-Jones form:

$$E_{vdW} = \sum_{i < j} \epsilon_{ij} \left[\left(\frac{R_{ij}^*}{r_{ij}} \right)^{12} - 2 \left(\frac{R_{ij}^*}{r_{ij}} \right)^6 \right], \quad (5)$$

where the atom type-dependent radius $R_{ij}^* = R_i^* + R_j^*$ and the atom type-dependent well depth $\epsilon_{ij} = \sqrt{\epsilon_i \epsilon_j}$.

Note that equation (5) is related to the $A/r^{12} - B/r^6$ representation of the Lennard-Jones equation through $A = \epsilon_{ij} (R_{ij}^*)^{12}$ and $B = 2\epsilon_{ij} (R_{ij}^*)^6$. Following the AMBER convention,³ atom pairs separated by one or two bonds are excluded from the vdW sum, as their interactions are already represented by the bond and angle terms. In addition, atom pairs separated by three bonds are scaled by a factor of one-half.

Parameter source. All bonded and nonbonded parameters were drawn from the GAFF2 force field (gaff2.dat, Version 2.2.20, March 2021) distributed with AmberTools 25. A subset was extracted covering the 17 atom types used in this work: c (sp² carbonyl C), c1 (sp C), c2 (sp² non-aromatic C), c3 (sp³ C), ca (aromatic C), ce (inner sp² C in conjugated systems), hc (H on aliphatic C), ha (H on sp²/sp C), hn (H on N), ho (hydroxyl H), n3 and n8 (sp³ N variants), o (carbonyl O), oh (hydroxyl O), and the

halogens `c1`, `br`, and `f`. We augmented the subset with bracketed section headers (`[bonds]`, `[angles]`, `[dihedrals]`, `[vdw]`) to simplify parsing the parameter file.

Molecular representation and atom typing. Each molecule is represented as an undirected graph whose nodes are atoms and whose edges are covalent bonds, stored as an adjacency list. Atoms store their element symbol, Cartesian coordinates, and an AMBER type (uninitialized at construction). Bonds are inferred pairwise, where a candidate atom pair is registered as bonded if their interatomic distance falls within $\pm 10\%$ of any tabulated reference value for that element pair (**Table A1**). The reference table covers C–C, C–H, C–N, C–O, N–H, O–H, C–Cl, and C–Br bonds.

Atom types are then assigned through a four-stage process:

1. Halogen atoms are assigned directly.
2. Oxygen atoms are typed as `oh` or `o` from their coordination number, and nitrogen atoms with three bonds are typed as `n8`.
3. Carbons are typed by examining the distances and elements of their neighbors. A C–C bond $< 1.28 \text{ \AA}$ is assigned as `c1` while a C–C bond of $1.28\text{--}1.45 \text{ \AA}$ is assigned as `c2`. A C=O bond is assigned as `c` while remaining carbon atoms are assigned as `c3`.
4. Lastly, hydrogens are assigned a type based on the type of atom they are bonded to.

Once typing is complete the adjacency list is used to return every pair, triplet, and quadruplet in the molecule. These tuples form lookup keys for bond, angle, and dihedral parameter maps, and are used to calculate the energy terms in equations 2-5.

Implementation and benchmark pipeline. Our code makes use of `armadillo` for vector and matrix operations. The build is managed through CMake, producing a single executable that accepts an XYZ file and a parameter file as command-line arguments. To establish a benchmark, each molecule was also processed using AmberTools 25. The electrostatic contribution was excluded from the comparison so that the four terms common to both implementations could be compared directly. Per-term energies from both implementations were saved to HDF5 file formats and accessed when making parity plots to summarize model results.

Results

Benchmarking against official AMBER Force Field

We validated our implementation against the reference AmberTools 25 pipeline, which itself comprises five stages: *obabel* for input file conversion, *antechamber* for atom-type and partial-charge assignment, *parmchk2* for parameter completion, *tleap* for topology construction, and *sander* for the single-point energy evaluation. For our benchmarking study, we included 40 different organic compounds, ranging from methane to amino acids, spanning the seven functional-group categories used in the RMSE analysis. Because our implementation omits the electrostatic term, we compare only the four common contributions - bond stretching, angle bending, dihedral torsion, and van der Waals - on a term-by-term basis. For each molecule we computed the signed residual 'custom - AMBER' for each of the four terms and aggregated those residuals into per-group root-mean-square errors (RMSE).

- **Parity across functional groups.** **Figure 1** shows parity plots for the total energy calculated by the AmberTools 25 pipeline and our custom implementation. Across the full benchmark set, points generally cluster tightly along the $y = x$ line, with the largest deviations confined to a handful of polar and heteroatom-containing molecules, while pure hydrocarbons sit essentially on top of the reference line.
- **Scaling with chain length.** For the straight-chain alkane series (methane to n-hexane), the absolute per-molecule energies grow approximately linearly with chain length in every term. Each successive CH_2 insertion adds new bonds, angles, and torsions to the enumerated tuples - and thus new degrees of freedom for stretching, bending, and rotation - as well as new atom pairs separated by four or more bonds that contribute to the van der Waals sum. The near-linear scaling

per CH₂ is consistent with the additive, pair-and-tuple-decomposed form of the AMBER functional. Fig especially highlights this trend.

- Clustering by chemistry:** When the results are grouped by category, several physically intuitive patterns appear.
 - Alkynes** show low dihedral energies, reflecting the rigidity imposed by the C≡C triple bond and the small number of rotatable single bonds in the test molecules.
 - Carboxylic acids and amino acids**, which contain heavier and more polarizable atoms (O, N), accumulate larger per-molecule energies in every term: heteroatoms introduce additional bonded environments (C–O, C–N, O–H, N–H) and additional vdW pairs with characteristically larger ϵ .
 - Halogenated species** similarly show inflated vdW contributions because of their large radii.
 - Pure **hydrocarbons** exhibit the smallest term-by-term deviations from the AmberTools reference, while the largest residuals concentrate on mixed-functional-group or conjugated molecules, where the appropriate GAFF2 atom type depends on a broader bonding environment than our four-stage typing routine inspects.
- Overall RMSE agreement.** Across the seven functional-group categories included in the RMSE analysis in **Figure 2**, (straight-chain alkanes, branched alkanes, alkenes, amines, halogenated hydrocarbons, alcohols, and carboxylic acids), the RMSE remains below 0.3 kcal/mol, and most are well below 0.1 kcal/mol.

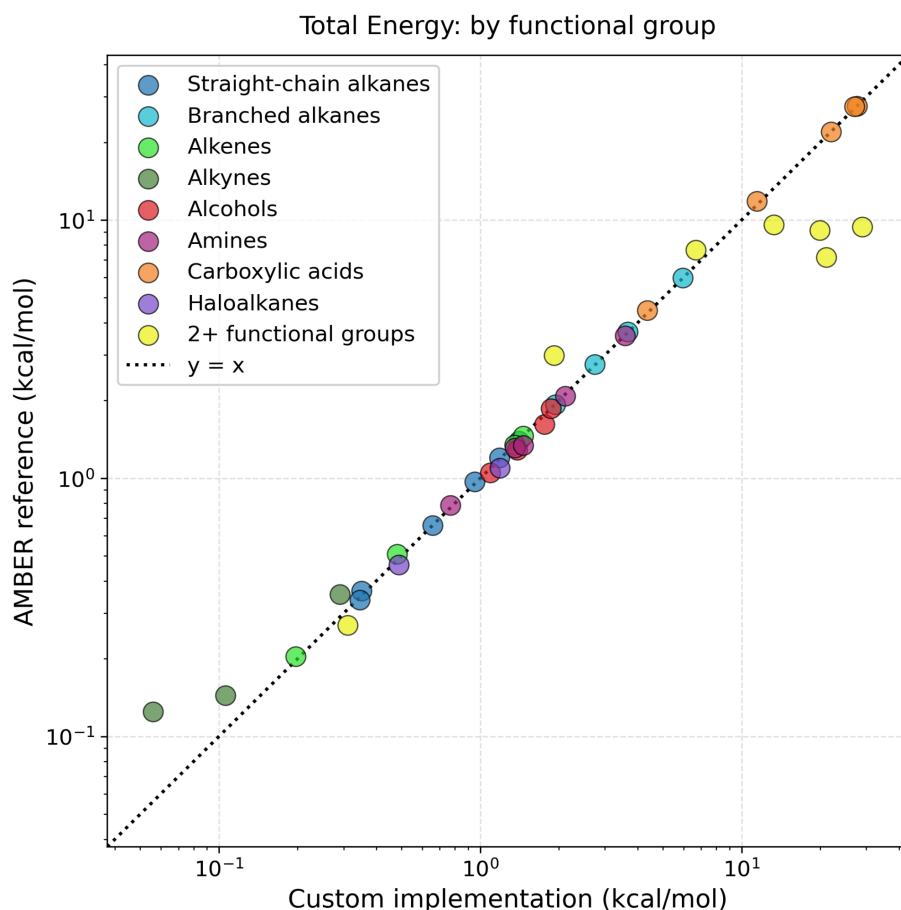


Figure 1. Parity plot of total energy (sum of four implemented terms: bond stretching, angle bending, torsion potential, and van der waals) for AMBER reference calculation from AmberTools 25 vs. our custom implementation in C++.

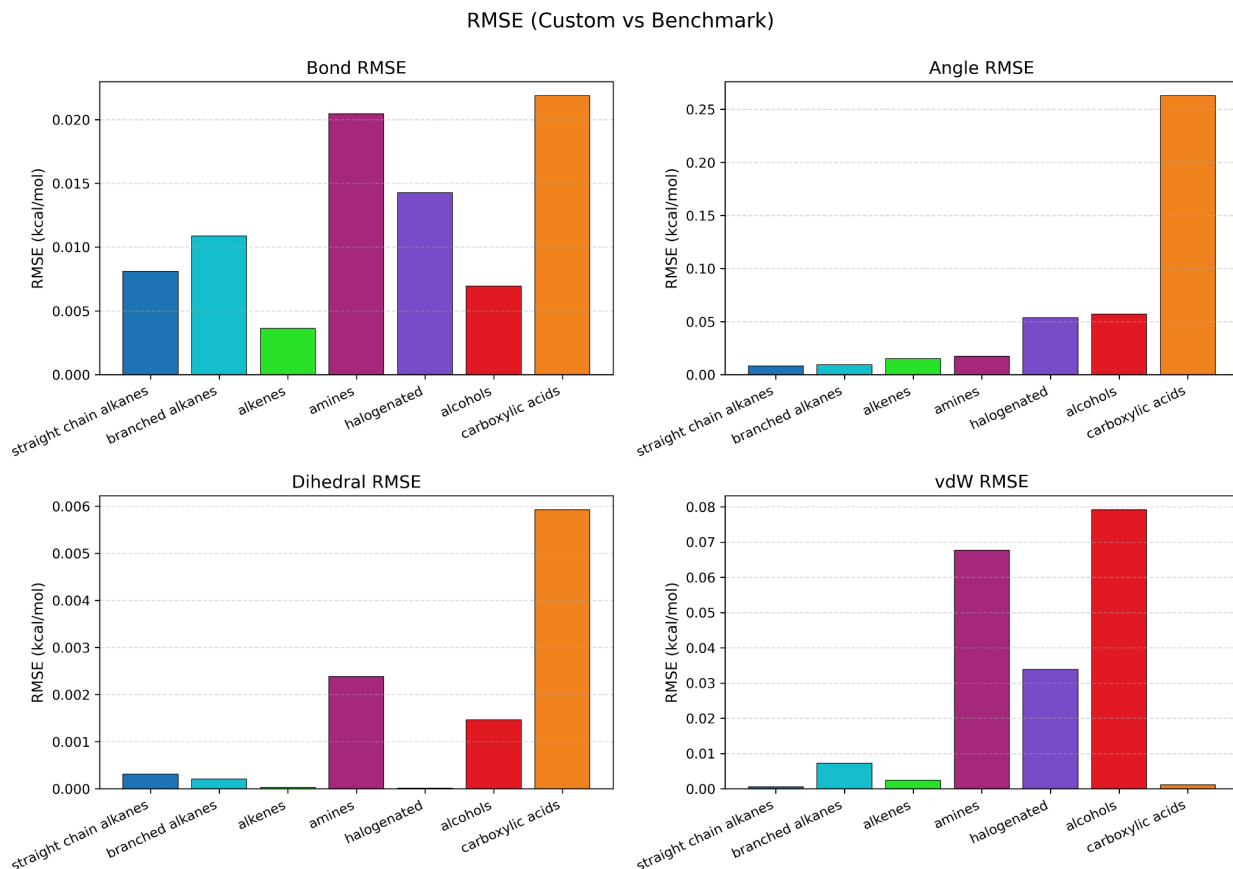


Figure 2. Root mean square error (RMSE) of each implemented energy term for seven classes of molecules.

Dihedral scan of 1-butene

To probe geometry dependence directly, we ran a scan along the $C(sp^3)-C(sp^2)$ dihedral of 1-butene from -180° to 180° in fixed increments and recomputed all four energy terms at each point. The results show three clear features (**Figure 3**).

1. The torsion energy oscillates with a 120° period, tracing the truncated Fourier form of equation (4); the maxima correspond to eclipsed geometries that frustrate the quantum-mechanical preference for $\sigma \rightarrow \pi^*$ hyperconjugation across the $C(sp^3)-C(sp^2)$ bond.
2. The van der Waals energy reaches its maximum when the methyl group on the sp^3 carbon is rotated into the plane of the sp^2 C-H bond, bringing CH_3 hydrogens into close contact with the sp^2 hydrogen and capturing the steric repulsion that pair-additive LJ terms are designed to encode.
3. The bond-stretching and angle-bending energies are essentially constant across the scan, as expected: the geometry change is a torsion about a single bond, and the harmonic stretches and bends are insensitive to it.

This decomposition shows that even with the limited 17-type atom typing supported here, the relative ordering of the four contributions and their geometric dependence are reproduced correctly.

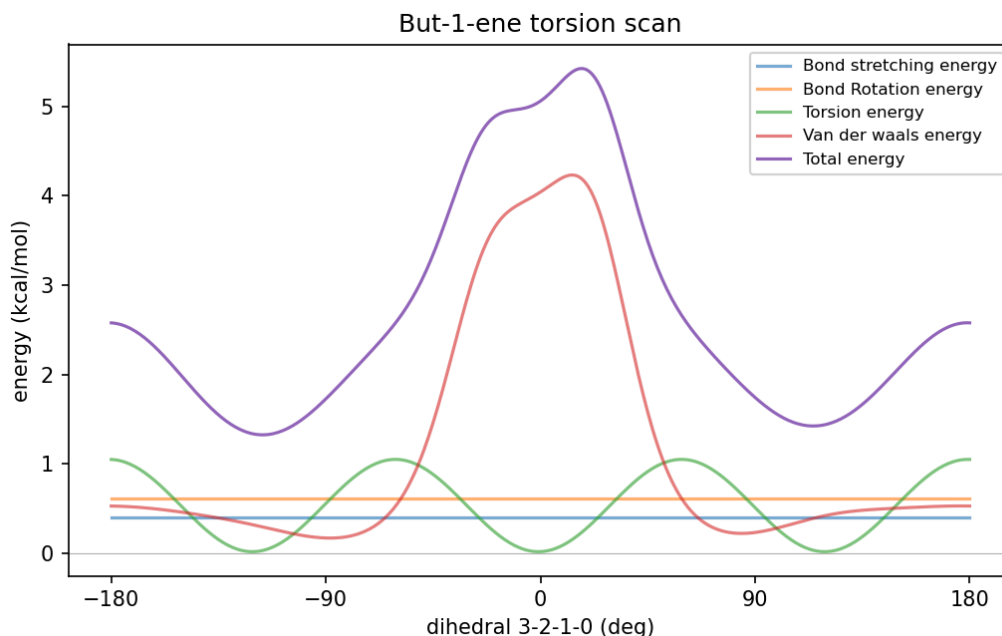


Figure 3. Torsion potential energy as a function of bond rotation angle

Discussion

The benchmark results indicate that the four implemented AMBER energy terms are computed correctly given a correct atom typing, and that the dominant source of residual disagreement with the AmberTools reference is the typing step itself rather than the energy kernels. Two observations support this:

1. The largest term-by-term deviations occur on molecules whose correct amber-type depends on bonding environments beyond an atom's immediate neighbors - for example, conjugated systems where a carbon should be typed ``ce`` rather than ``c2``, or polar molecules with multiple competing heteroatom environments. Our typing routine uses local neighbor distances and elements only, so it cannot always reproduce the multi-step, substructure-aware rules used by ``antechamber``.
2. Conversely, pure hydrocarbons - for which the local rules in ``type_carbon_backbones`` and ``type_hydrogens`` are essentially complete - show the smallest deviations and the best parity slopes.

We made the typing rules as general as possible within a four-stage local scheme, but it is difficult to anticipate every bonding context with a finite test set; extending coverage to aromatic systems or more elaborate heteroatom environments would require substructure-pattern matching closer to what ``antechamber`` performs internally.

The dihedral scan further illustrates the physical content captured by the four implemented terms. Torsion energies follow the cosine series that fits the underlying electronic preference for orbital overlap and hyperconjugation, vdW energies encode steric repulsion at eclipsed geometries, and the constancy of the stretching and bending terms across the scan confirms that they respond, as they should, only to changes in bond lengths and bond angles. Together, the parity benchmarks and the scan validate both the per-term implementation and the qualitative physics of the AMBER functional form as realized in our code.

Conclusion

We successfully calculated four of five energy terms in the AMBER force field in C++. Several aspects of our implementation enabled accurate calculation of the force field terms.

- **Parameter file:** extracting bonded and nonbonded parameters from AmberTools 25 and augmenting the file with bracketed section headers to simplify file parsing.
- **Data structures:** representing molecules as undirected graphs, where nodes are atoms and edges are bonds (`MoleculeGraph``). In turn, atoms store their element symbol, coordinates, and AMBER type. The latter attribute was important in allowing us to create a tuple mapping every pair, triplet, and quadruplet in the molecule to their respective AMBER parameters.
- **Atom typing routine:** four-stage process for assigning atom types in the molecule, starting with halogen assignment and ending with hydrogen assignment.
- **Benchmarking:** implementing an AmberTools 25 pipeline for calculating reference energies to benchmark our calculated energies to. Analyzing results as a parity plots for each energy term allowed quick iteration on different molecule types and which terms carried the most error.

Together, these design choices allowed us to systematically test the force field energy calculations for various classes of molecules. The major source of error between our implementation and reference values originates from the appropriate GAFF2 atom type depending on a broader bonding environment than our four-stage typing routine currently supports.

To summarize, force fields serve as the critical bridge between quantum mechanical properties and macroscopic observables, enabling simulations to scale from a handful of atoms to systems containing millions. Rather than solving the Schrödinger equation explicitly, which becomes computationally intractable beyond a few hundred atoms, force fields approximate the potential energy surface using a set of classical, analytical expressions. This trade-off between accuracy and computational cost is what makes molecular dynamics simulations of biological macromolecules, polymers, and materials feasible. The AMBER force field achieves this through just five analytical energy terms: bond stretching, angle bending, torsional rotation, van der Waals interactions, and Coulombic (electrostatic) interactions. Despite the apparent simplicity of this functional form, the real complexity lies in the extensive parameter library - thousands of carefully fitted values for charges, force constants, and equilibrium geometries that have been refined against high-level quantum mechanical calculations and experimental data. Accurate atom typing is crucial to make this parameterisation useful: each atom is assigned a type based on its element, hybridisation state, and local chemical environment, which in turn determines which parameters are applied. While this system yields excellent accuracy for well-characterised functional groups, it inherently limits the transferability of the force field. Phenomena such as bond breaking and formation, charge polarisation in response to the local environment, and charge-transfer effects cannot be captured within the fixed-charge, harmonic framework that classical force fields rely upon.

Future work

1. Atom-type detection should be extended to cover aromatic, cyclic, and conjugated systems - motifs common in pharmaceuticals and biomolecules but unsupported by simple connectivity-based typing. Graph traversal algorithms, implemented via C++ libraries such as the Boost Graph Library, can enable systematic ring detection, aromaticity assignment, and identification of conjugation.
2. The energy framework can be extended to estimate reaction energies as the difference in potential energy between products and reactants. This also serves as a natural stepping stone toward hybrid QM/MM methods.
3. Adding force evaluation and geometry optimisation would transform the codebase into a capable molecular dynamics simulator. Forces follow analytically as the negative gradient of the potential energy, and can drive geometry optimisation via steepest descent. Introducing a time integrator such as velocity Verlet then enables molecular dynamics, opening access to conformational sampling and equilibrium ensemble properties.

Contributions

- Matthew:
 - **Data pipeline and input representations:** structure for augmented .xyz files/parameter files and code for parsing input .xyz files. Developed the initial code for extracting and parsing a hydrocarbon parameter file for linear alkanes.
 - **Proof of concept validation:** debugging the pipeline to ensure correct assignment of atomic neighbors and coordinates
 - **Report and presentation:** proposed to summarize data as parity plots + initial Python wrapper for running scripts in batch and making the parity plots; slide design; background and codebase overview of final presentation; background + objectives + methodology in final report
- Jahnavi:
 - Code for detecting bonds from the atom's coordinates, additionally finding the atom triplets/ quadruplets for calculating bond angle and dihedral angle
 - Code for atom type detection based on the atom's element, bond order, and the atom's neighborhood
 - Code for calculating the 4 force field terms using the parameters map, atom-type and bond length/ bond angle/ dihedral angle
 - Running the benchmark testing against the actual AMBER force field on 40 different organic molecules using the AmberTools25 package, and analyzing the results/ errors
 - Exploring the use of the force field to study the changes in energies with changes in geometry, using 1-butene as an example

Code availability

The code used to generate the plots shown in this project is available through GitHub:

<https://github.com/matthew-j-liu/AMBER-ff>

References

1. Darden, T., York, D. & Pedersen, L. Particle mesh Ewald: An $N \cdot \log(N)$ method for Ewald sums in large systems. *J. Chem. Phys.* **98**, 10089–10092 (1993).
2. Batatia, I. *et al.* A foundation model for atomistic materials chemistry. Preprint at <https://doi.org/10.48550/arXiv.2401.00096> (2025).
3. Cornell, W. D. *et al.* A Second Generation Force Field for the Simulation of Proteins, Nucleic Acids, and Organic Molecules. *J. Am. Chem. Soc.* **117**, 5179–5197 (1995).
4. Halgren, T. A. Merck molecular force field. I. Basis, form, scope, parameterization, and performance of MMFF94. *J. Comput. Chem.* **17**, 490–519 (1996).
5. Ponder, J. W. *et al.* Current Status of the AMOEBA Polarizable Force Field. *J. Phys. Chem. B* **114**, 2549–2564 (2010).

Appendix

Section A1. Code Overview

The implementation is written in C++ and built with CMake, producing a single executable, ``amber_ff``, that takes an XYZ geometry file and a GAFF2 parameter file as command-line arguments and prints the per-term and total potential energies to stdout. The source is organized so that each stage of the pipeline lives in its own translation unit, and a small set of plain-old-data structures carries information between stages.

Execution Pipeline (or just include a diagram here)

Figure A1 in the **Appendix** shows a flowchart that summarizes the execution pipeline. Here, we describe each step of the workflow in detail. ``main()`` wires the four stages of the calculation together:

1. **Parameter loading.** ``load_params()`` (in ``read_params.cpp``) parses the AMBER parameter file ``selected_atoms.dat``, with ``[bonds]``, ``[angles]``, ``[dihedrals]``, and ``[vdw]`` sections. One small helper per section (``parse_bond_row``, ``parse_angle_row``, ``parse_dihedral_row``, ``parse_vdw_row``) converts each row into the appropriate parameter struct and inserts it into the ``ForceField`` container.

2. **Geometry and connectivity.** ``read_xyz()`` (in ``read_xyz.cpp``) reads atom elements and Cartesian coordinates into a ``MoleculeGraph``, then calls ``assign_bonds()`` to infer covalent connectivity and ``assign_amber_types()`` to label each atom with its GAFF2 type. Bond inference uses a hard-coded reference table, ``BOND_LENGTHS`` (in ``parsing.hpp``), covering C-C, C-H, C-N, C-O, N-H, O-H, C-Cl, and C-Br bonds. As discussed in the *Methodology* section, a pair of atoms is treated as bonded if their interatomic distance falls within $\pm 10\%$ of any tabulated reference for that element pair.

3. **Atom typing.** As detailed earlier in the *Methodology* section.

4. **Energy evaluation.** Four high-level routines in ``ff.cpp``: ``calculate_bonds_term``, ``calculate_angles_term``, ``calculate_dihedrals_term``, and ``calculate_vdw_term`` - enumerate the relevant atom groups from the molecular graph and accumulate the contributions. Group enumeration is provided by ``MoleculeGraph::find_all_bonds()``, ``find_all_bond_angle_triplets()``, and ``find_all_torsion_quadruplets()``, which walk the adjacency list to return pairs, triplets, and quadruplets, respectively. Each tuple is converted into an atom-type key used to look up parameters in the ``ForceField``; the dihedral lookup also tries X*-wildcard keys at the terminal positions. Low-level kernels (``bond_stretching_energy``, ``bond_rotation_energy``, ``bond_torsion_energy``, ``vdw_energy``) implement the functional forms in equations (2)–(5), and the geometric primitives (``bond_length``, ``angle_deg``, ``dihedral_angle``) live in ``geom_util.cpp`` and use Armadillo vectors throughout.

Running the Code

Building: the build is managed with CMake and depends only on Armadillo:

```
bash
```

```
cmake -B build && cmake --build build
```

This produces the `amber\_ff` executable used in all benchmarks.

Section A2. Additional tables and figures

Table A1. Reference table of bond lengths. Atom pairs within 10% of their respective values are considered bonded.

Element Pair	Reference Lengths (Å)	Bond orders
C-C	1.2095, 1.3185, 1.3349, 1.5088, 1.5354	c1-c1; c1-c2; c2-c2; c2-c3; c3-c3
C-H	1.0962	c3-hc
C-N	1.1619, 1.2809, 1.3967, 1.4626	c1-n1; c2-n2; c-n; c3-n3
C-O	1.2190, 1.2249, 1.3206, 1.3423, 1.3556, 1.4242	c-o; c2-o; c3-o; c2-oh; c-oh; c3-oh
N-H	1.0095, 1.0120, 1.0130, 1.0190	hn-na; hn-nh; hn-n; hn-n3
O-H	0.9725	ho-oh
C-Br	1.9054, 1.9805	br-c2; br-c3
C-Cl	1.7415, 1.8092	c2-cl; c3-cl

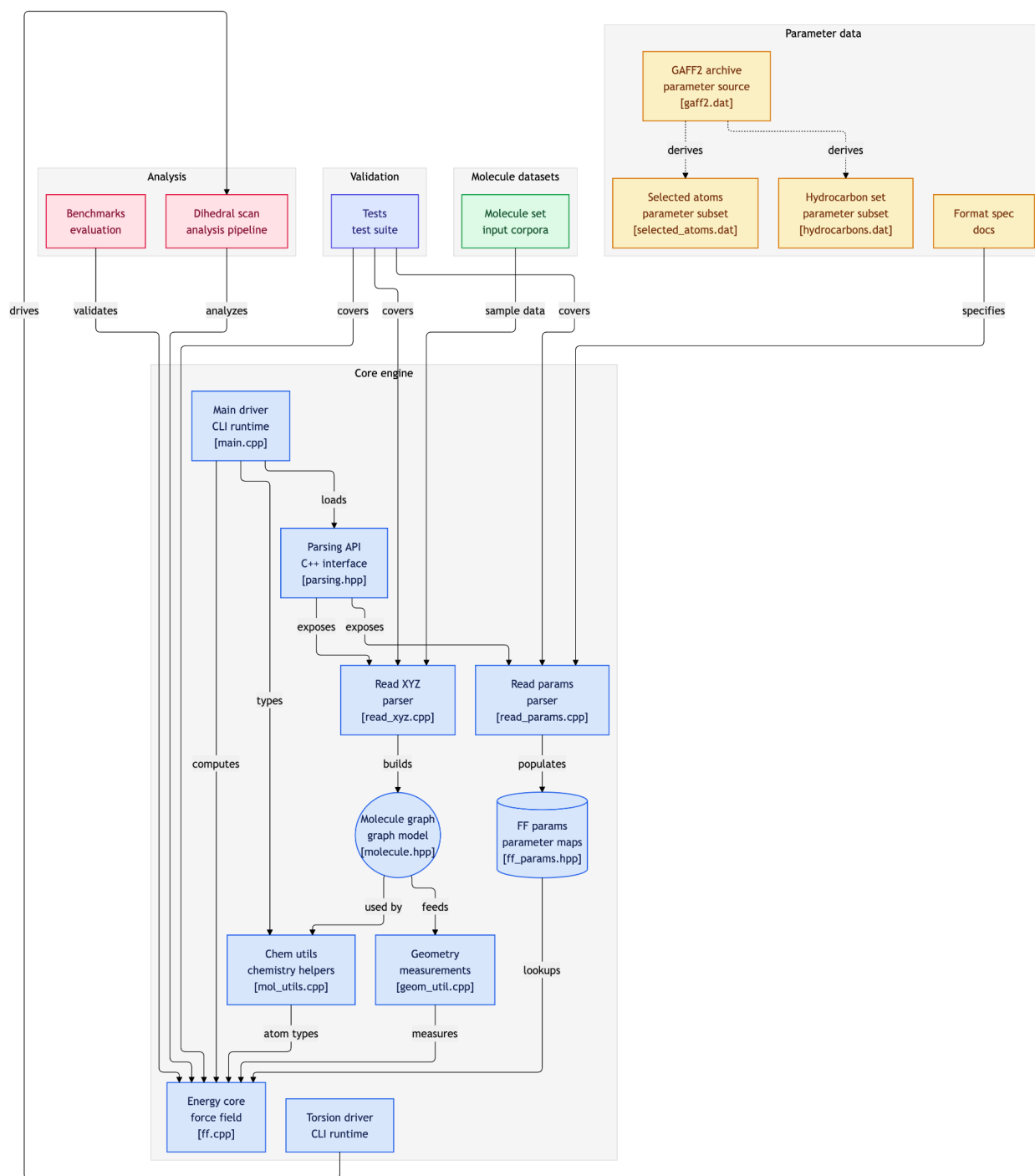


Figure A1. Overview of our C++ pipeline for calculating AMBER force field terms. Image generated with GitDiagram.